

Applied Deep Learning

Omri Azencot

The Stein Faculty of Computer and Information Science
Ben-Gurion University of the Negev

Sequence Neural Models



Motivation



Motivation

Chromatic (ascending):

5

S

do di re ri mi fa fi sol si la li ti do di re ri

B

do di re ri mi fa fi sol si la li ti do di re ri

9

S

mi fa fi sol si la li ti do

B

mi fa fi sol si la li ti do

★☆☆☆☆ One Star

By Joe Watson - December 14, 2014

There were no wolves in the movie.

0 of 3 people found this review helpful

A Bug's Life (1998)

★★★★★ Five Stars

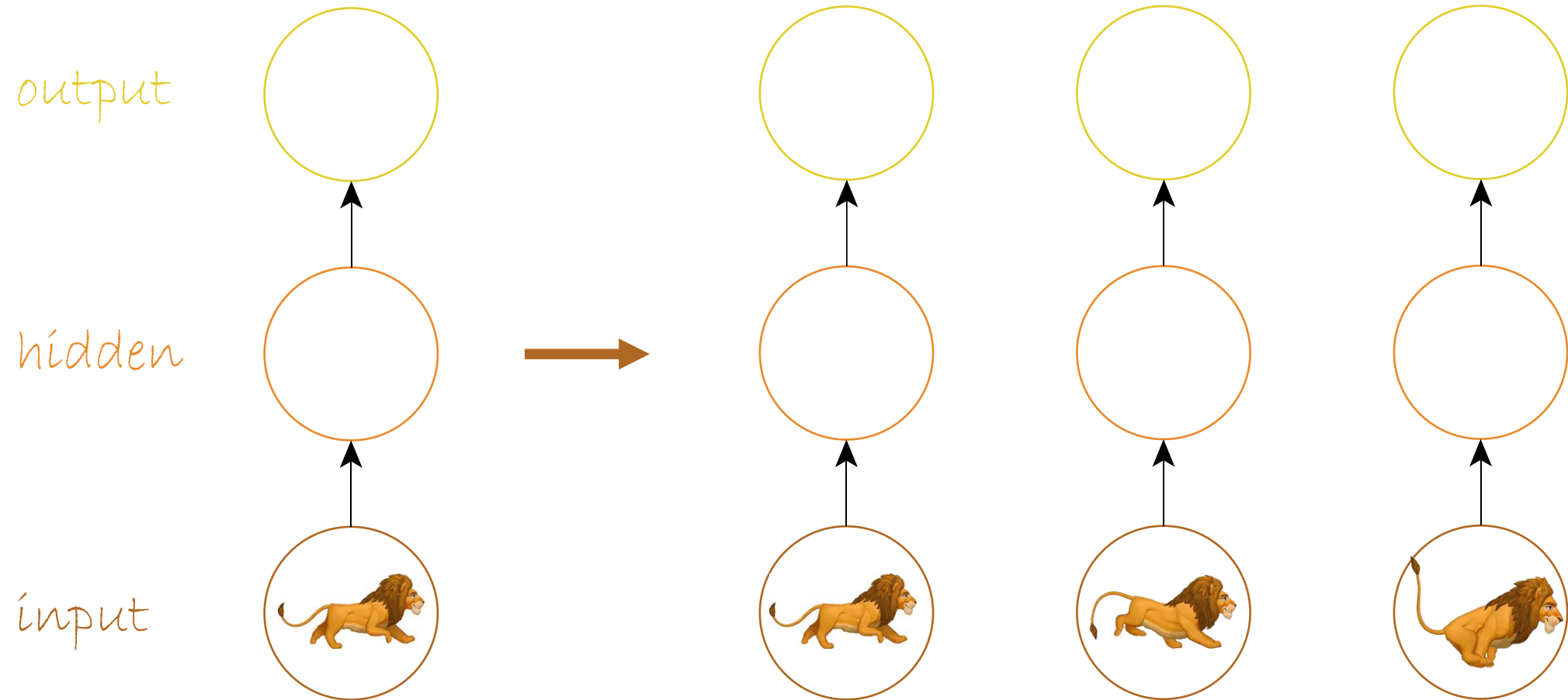
By colton - March 19, 2015

Amazon Verified Purchase

I never knew what struggles bugs went through.... changed my life



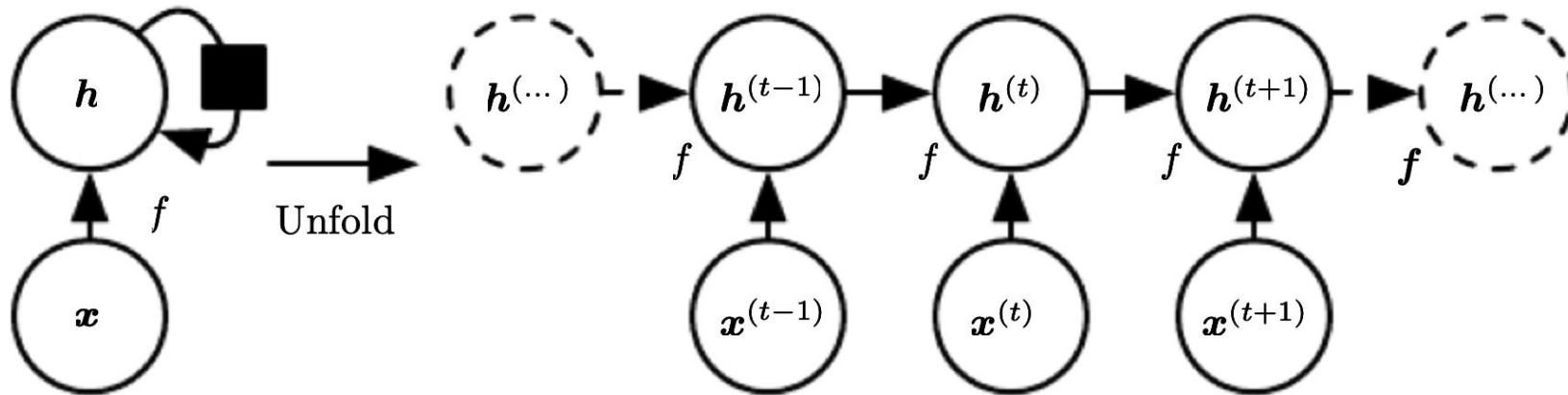
Multilayer vs. Sequential Networks



Unfolding Computational Graphs

state $\longrightarrow h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta)$

$\uparrow$ input



Recurrent Neural Networks (RNN)

state

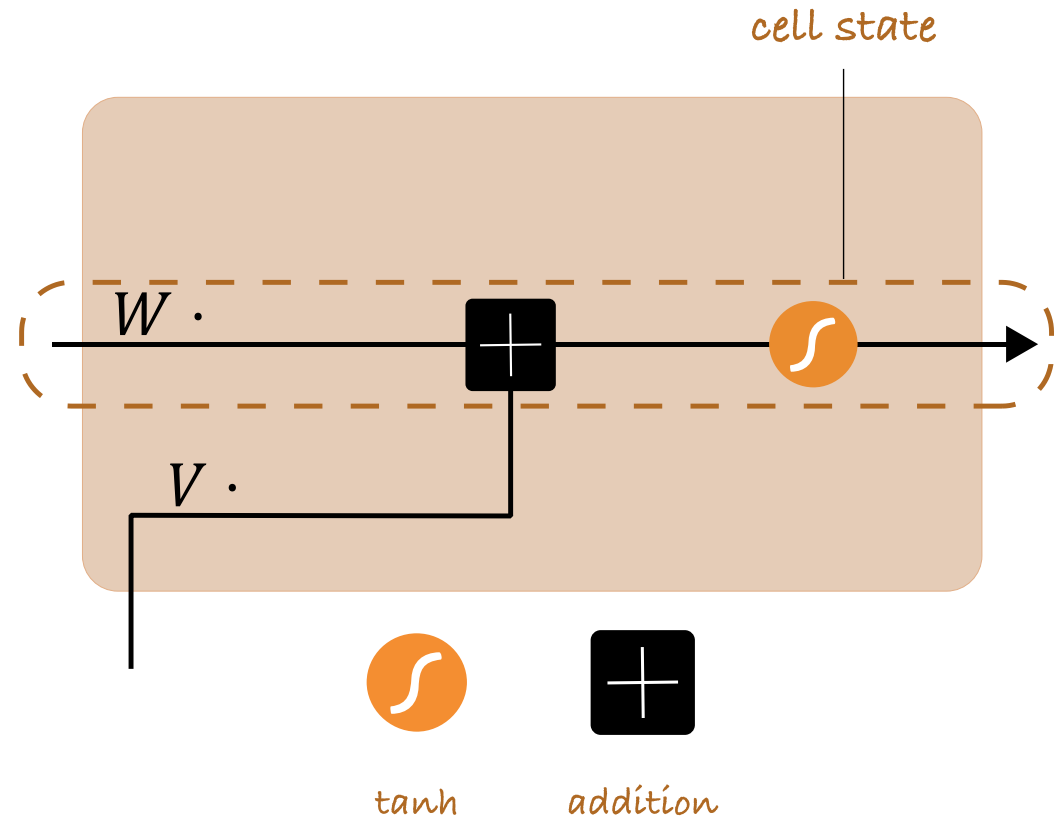
nonlinearity

input

$$h_t = \sigma(W h_{t-1} + V x_t)$$

hidden-to-hidden

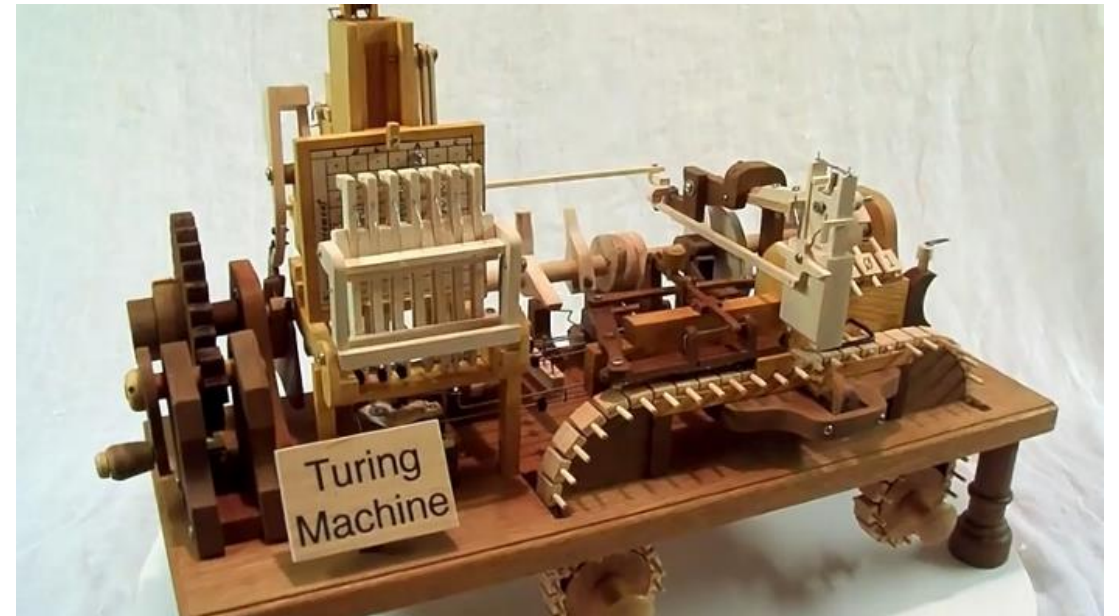
input-to-hidden



RNNs as Universal Approximators

$$h_t = \sigma(W h_{t-1} + V x_t)$$

↑ ↑
state input



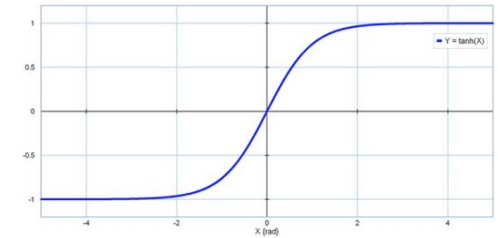
Equations of RNNs

$$z_t = \textcircled{b} + \textcircled{W}h_{t-1} + \textcircled{V}x_t$$

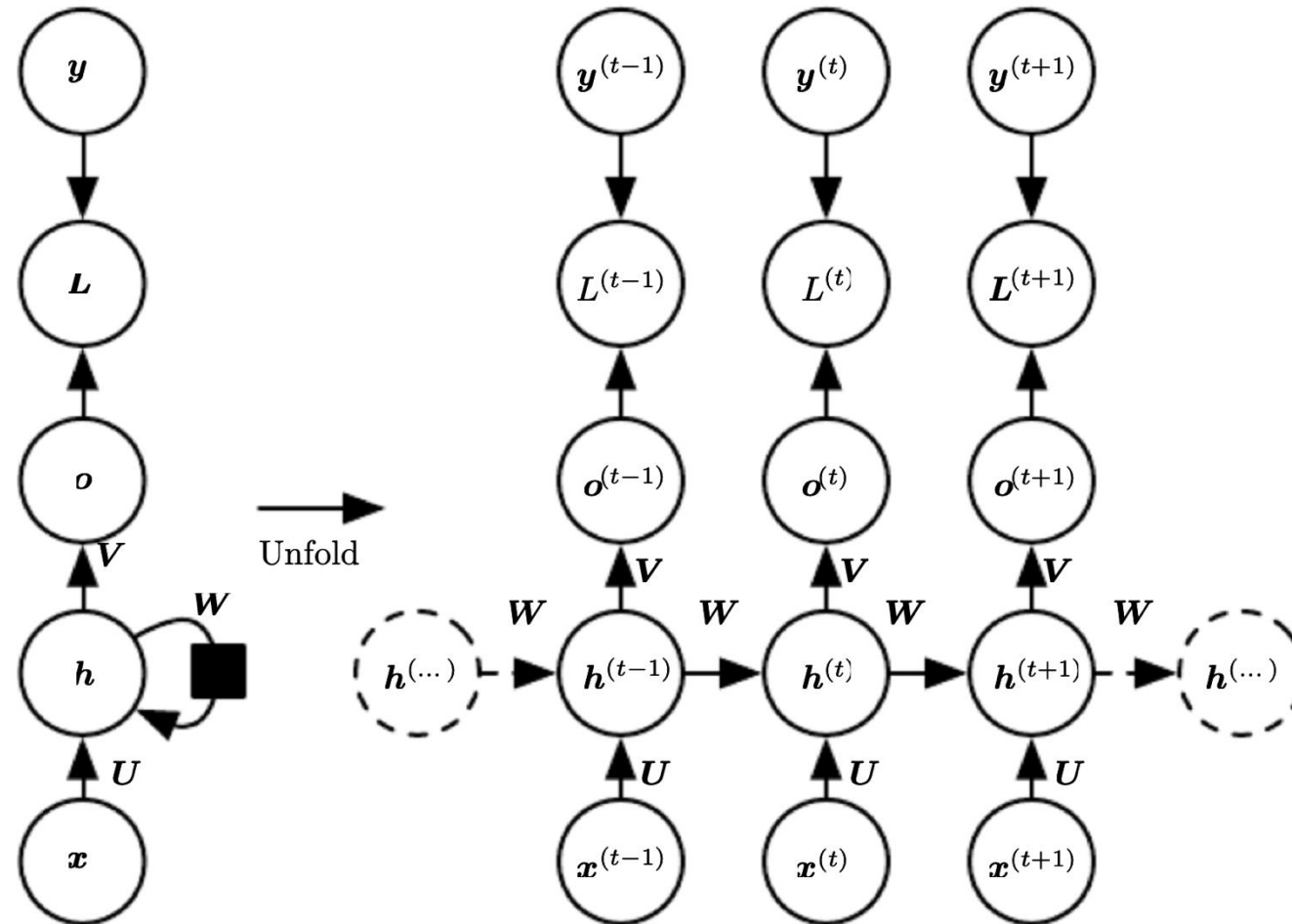
state $\longrightarrow$ $h_t = \sigma(z_t)$

$$o_t = \textcircled{c} + \textcircled{U}h_t$$

prediction $\longrightarrow$ $\tilde{y}_t = \rho(o_t)$

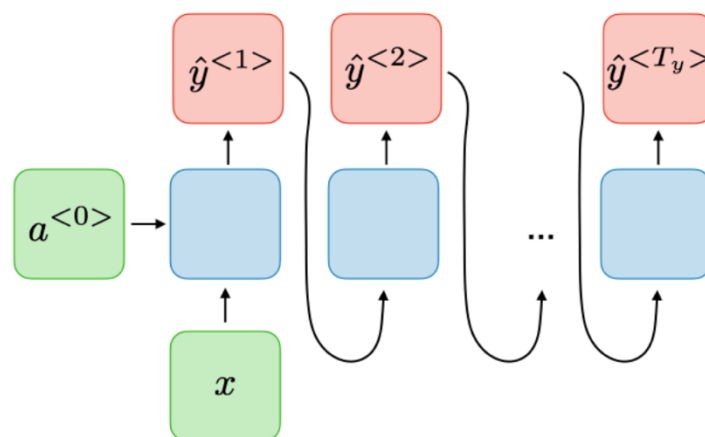


Computational Graph of RNNs



Design Patterns

One-to-many
 $T_x = 1, T_y > 1$



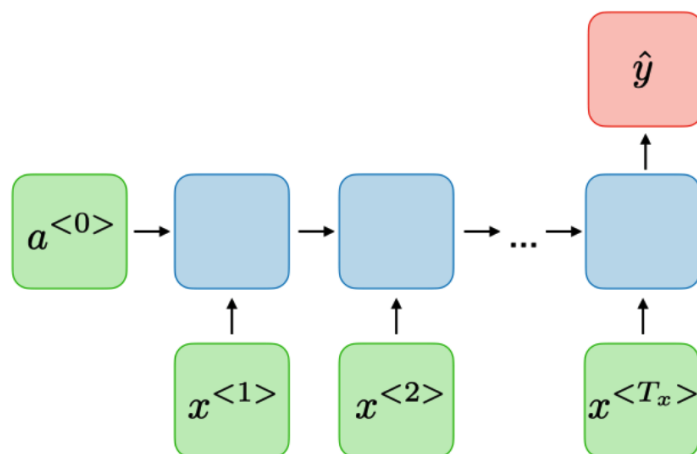
Video Generation



Design Patterns

Sentiment Analysis

Many-to-one
 $T_x > 1, T_y = 1$



By Joe Watson - December 14, 2014

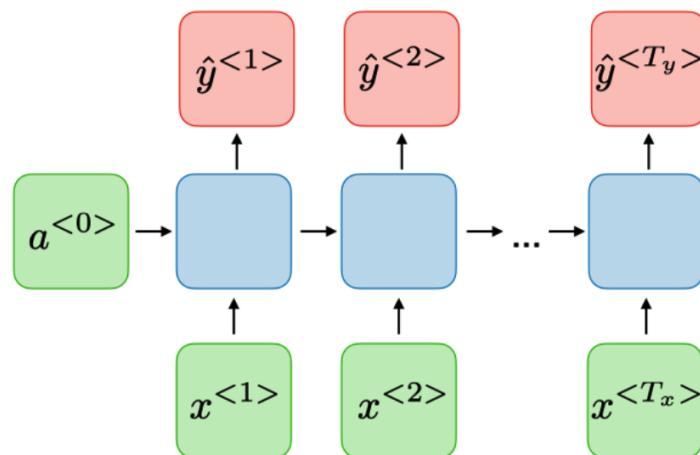
There were no wolves in the movie.

0 of 3 people found this review helpful



Design Patterns

Many-to-many
 $T_x = T_y$



Stock Prediction

S&P 500

3,974.54

+3,839.05 (2,833.46%) ↑ all time

26 Mar, 17:09 GMT-4 · Disclaimer

INDEXSP: .INX

+ Follow

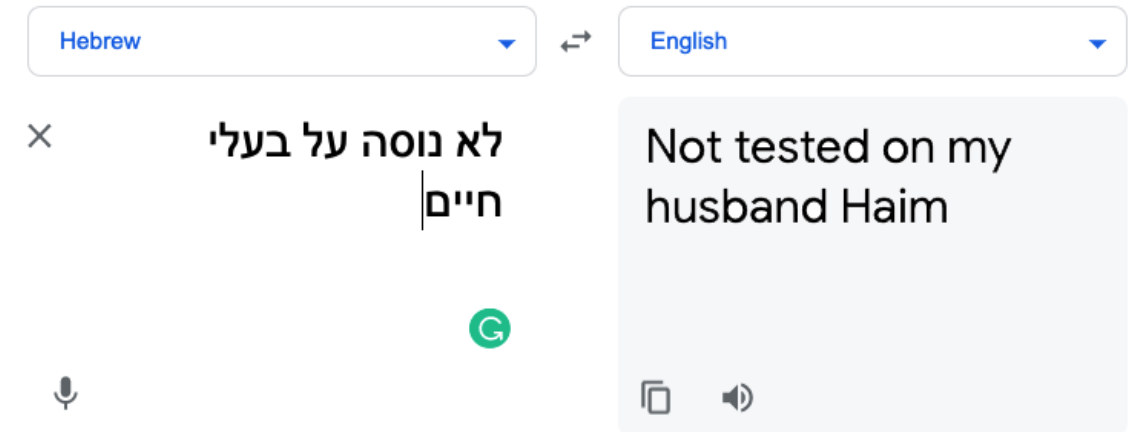
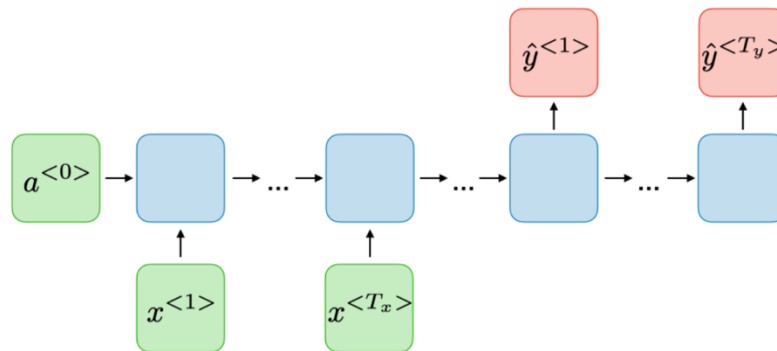
1 day | 5 days | 1 month | 6 months | ytd | 1 year | 5 years | max



Design Patterns

Text Translation

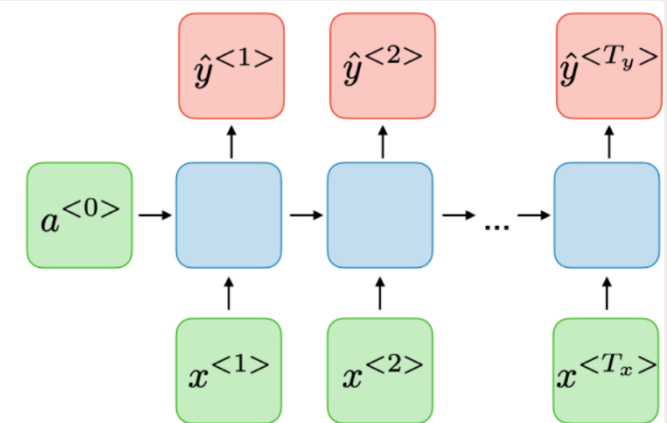
Many-to-many
 $T_x \neq T_y$



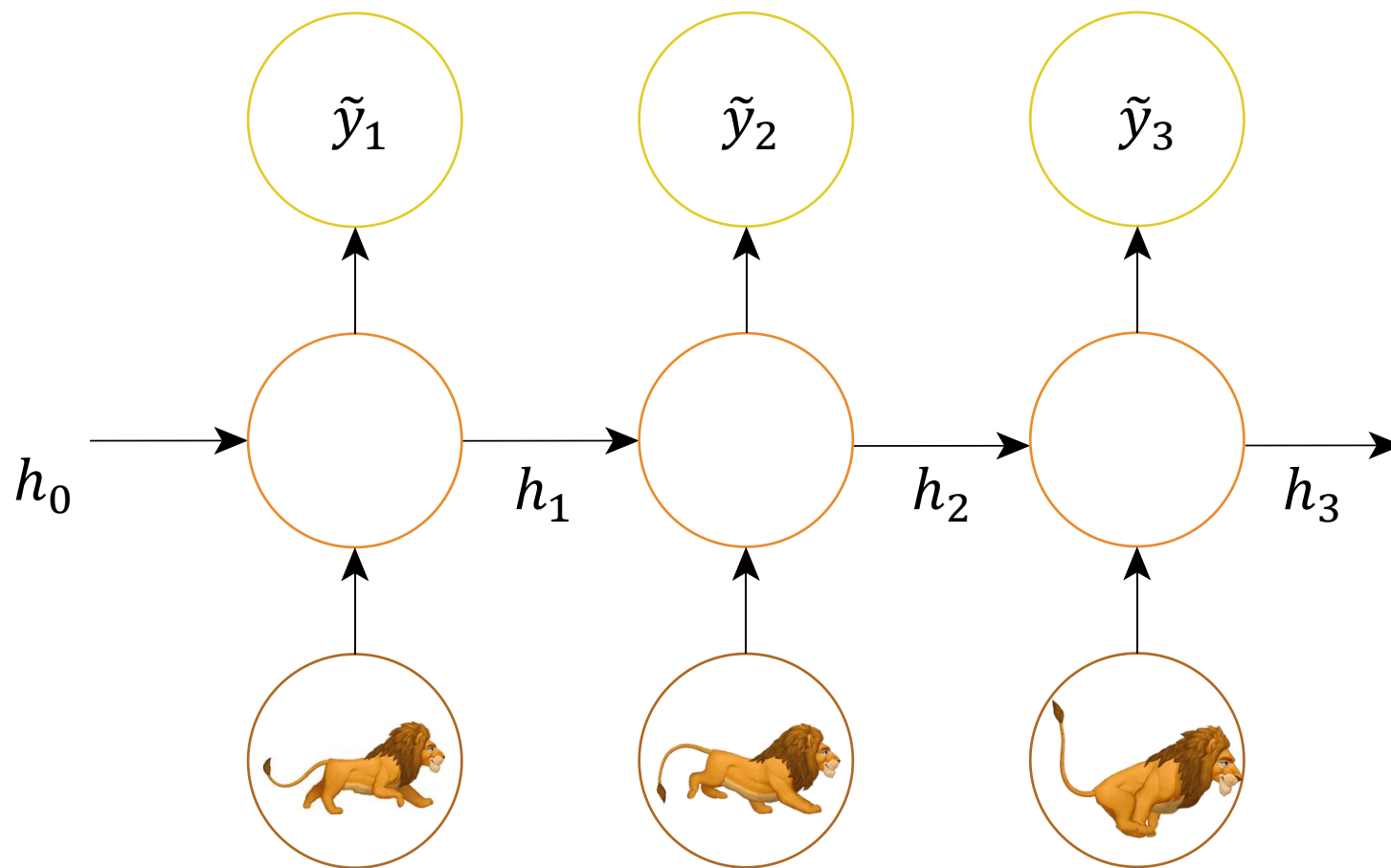
The Loss Function

Denote $L^{(t)}$ the **NLL** of $y^{(t)}$ given $x^{(1)}, \dots, x^{(t)}$, then

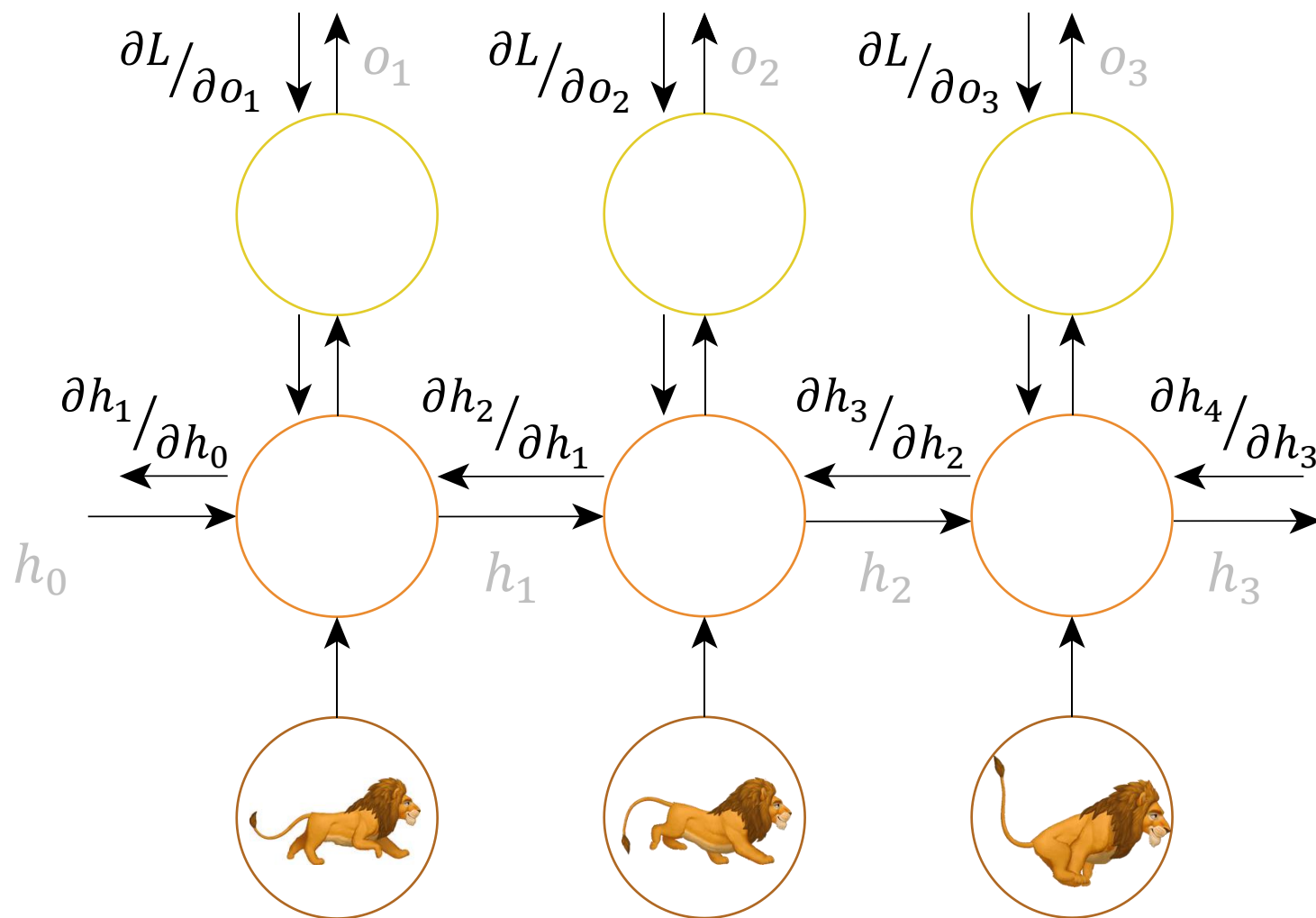
$$\begin{aligned} L(\{x^{(j)}\}, \{y^{(j)}\}) &= \sum_t L^{(t)} \\ &= - \sum_t \log p_{\text{model}}(y^{(t)} | \{x^{(1)}, \dots, x^{(t)}\}) \end{aligned}$$



The Forward Pass



The Backward Pass



The Backward Pass

$$\begin{array}{rcl} z_t & = & b + Wh_{t-1} + Vx_t \\ h_t & = & \sigma(z_t) \\ o_t & = & c + Uh_t \\ \tilde{y}_t & = & \rho(o_t) \\ L_t & = & -\log(\hat{y}_t \cdot y_t) \end{array}$$

Derivatives of o_t and h_t :

$$(\nabla_{o_t} L)(i) = \frac{\partial L}{\partial L_t} \frac{\partial L_t}{\partial o_t(i)} = \hat{y}(i) - 1$$

$$\nabla_{h_t} L = W^T \text{diag}(1 - h_{t+1}^2) \nabla_{h_{t+1}} L + U^T \nabla_{o_t} L$$



The Backward Pass

Derivatives wrt c , b , V , U , and W :

$$\nabla_c L = \sum \nabla_{o_t} L$$

$$\nabla_b L = \sum \text{diag}(1 - h_t^2) \nabla_{h_t} L$$

$$\nabla_U L = \sum \nabla_{o_t} L h_t^T$$

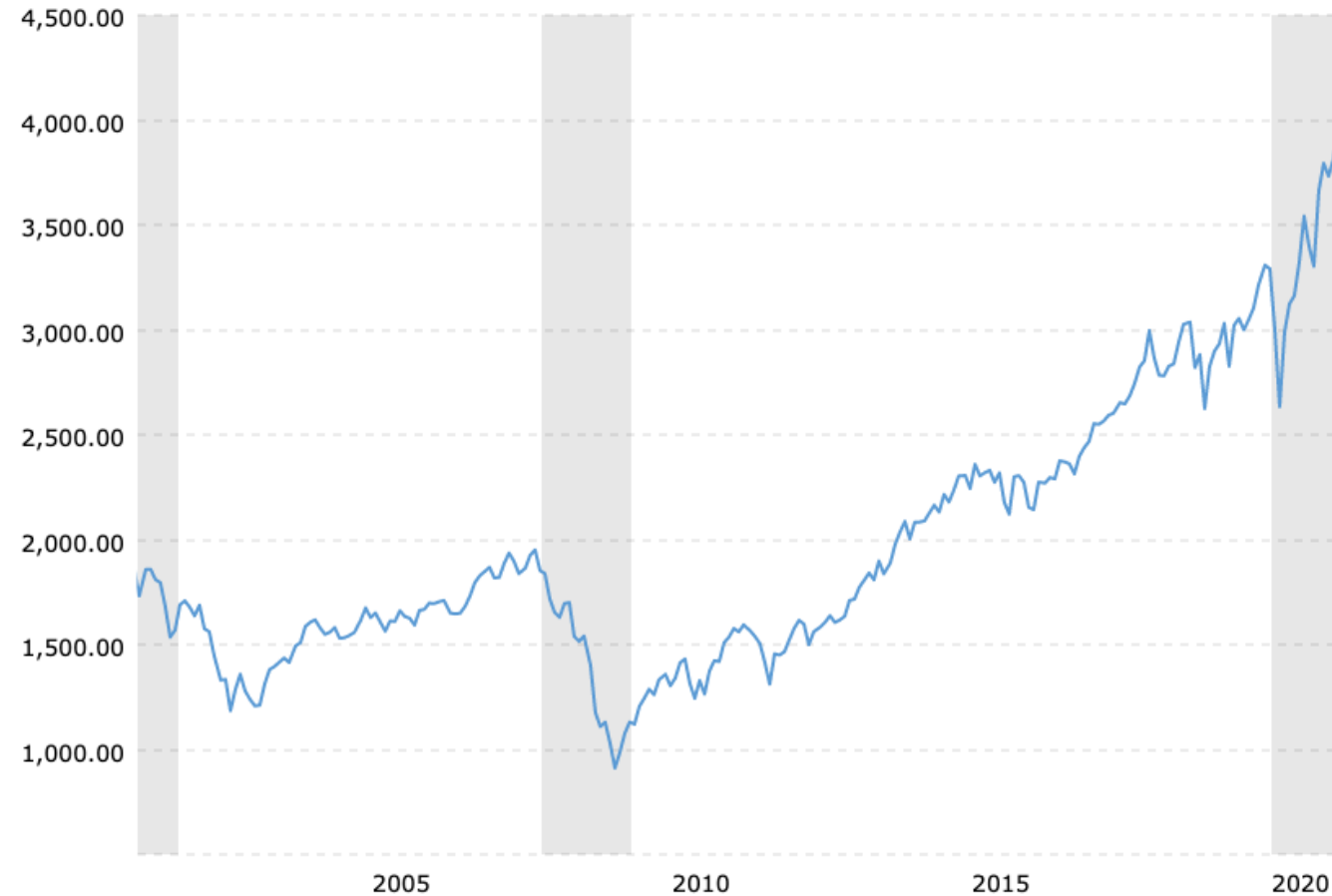
...



Modeling Long-Term Dependencies



Long-Term Dependencies



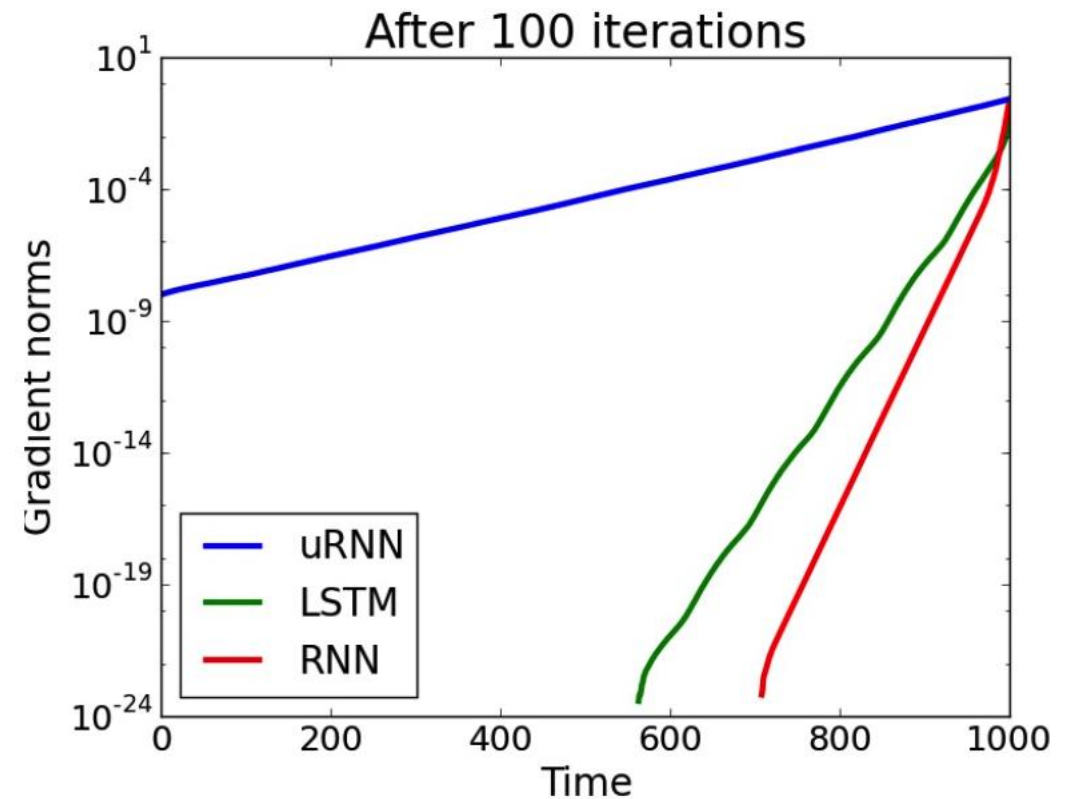
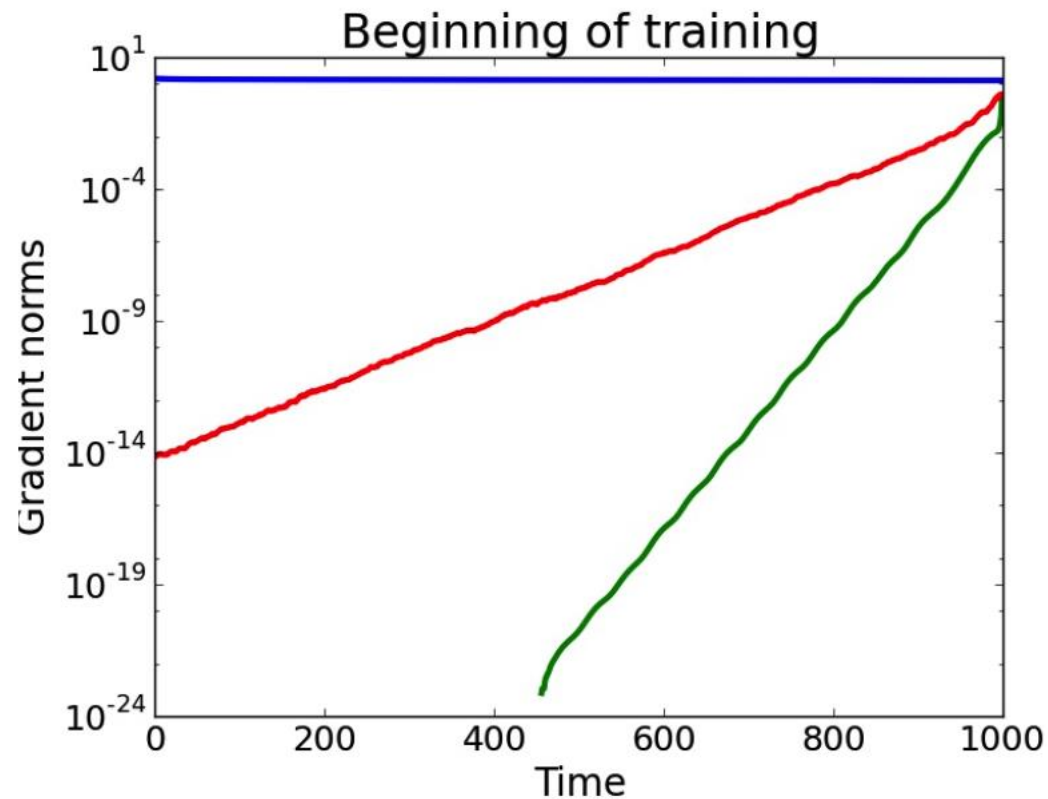
The Copy Task

Inputs and outputs of size $T + 2K$ over an **alphabet** of size L

Input: $\overbrace{92836}^K \text{-----} : \text{-----}$
Output: $\text{-----} \underbrace{\text{-----}}_T 92836$

T
→ what happens when $T \rightarrow \infty$?

Exploding and Vanishing Gradients



A Linear Recurrent Neural Network

Consider the following recurrent model

$$\text{state} \longrightarrow h_k = W h_{k-1}$$

hidden-to-hidden

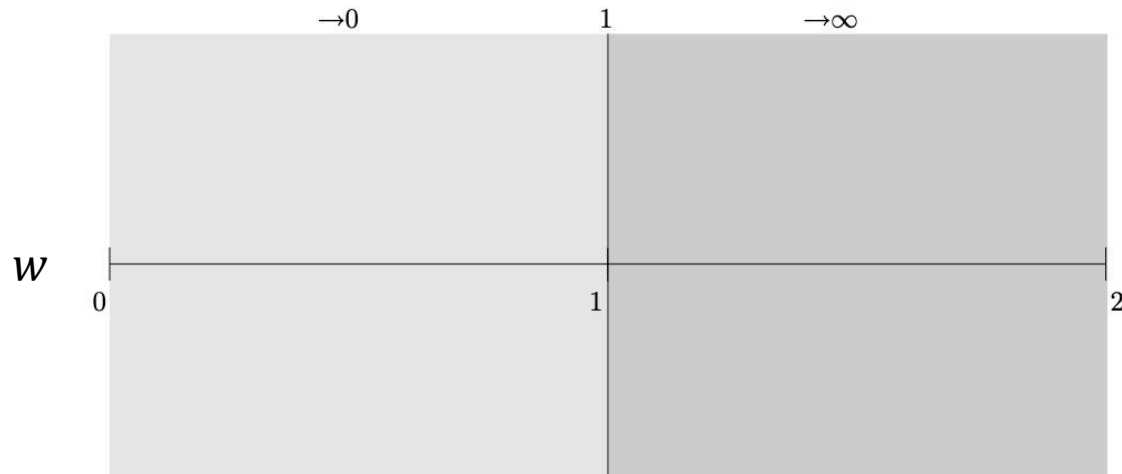
Simpler form:

$$h_k = W^k h_0$$

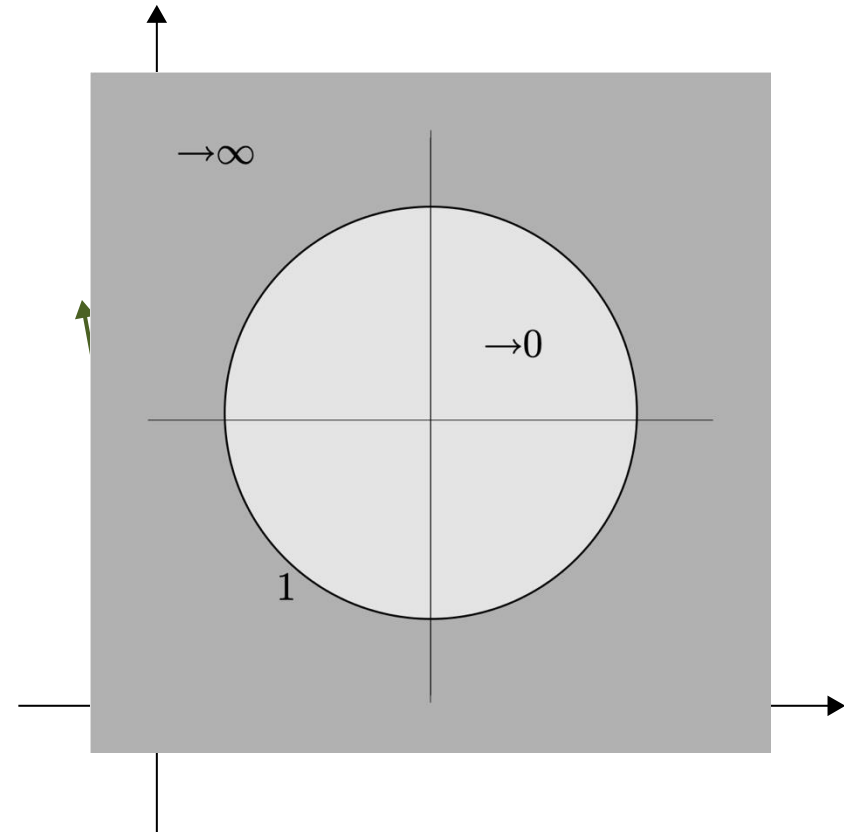
Exponentiating Numbers

If $w \in \mathbb{R}$, then

$$h_k = w^k h_0$$



If $w \in \mathbb{C}$, then



RNN Analysis via its Spectrum

Eigendecomposition

$$Wv = \lambda v$$

↑ ↑
eigenvector eigenvalue

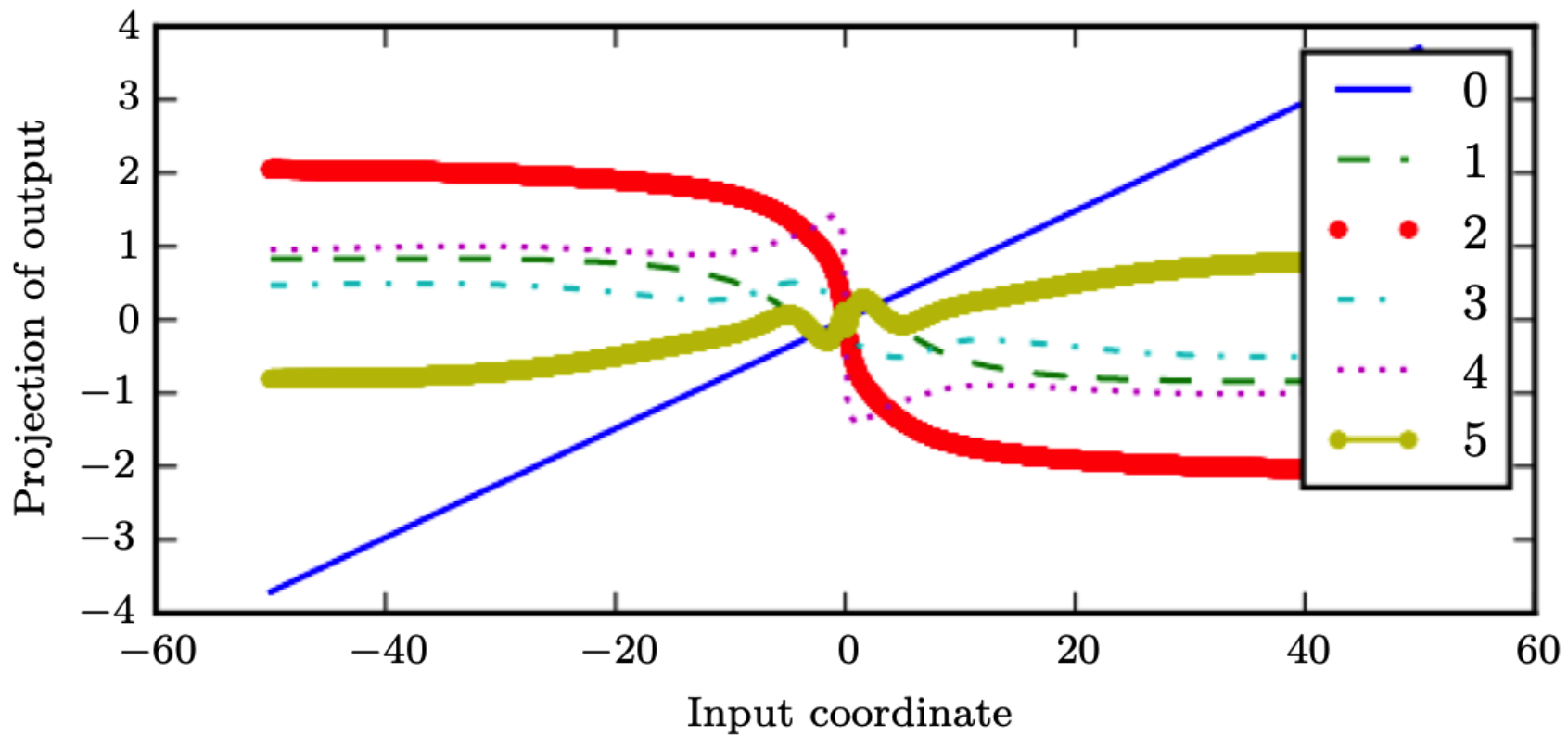
An eigenvector $v_{j,0}$ path

$$v_{j,k} = \lambda^k v_{j,0}$$

↑
 $h = \sum_j \tilde{h}_j v_j$



General RNN Models



$$\begin{aligned} h_t &= \sigma(b + Wh_{t-1} + Vx_t) \\ L_t &= L(h_t) \end{aligned}$$

General RNN Models

L denotes the **loss** function, then

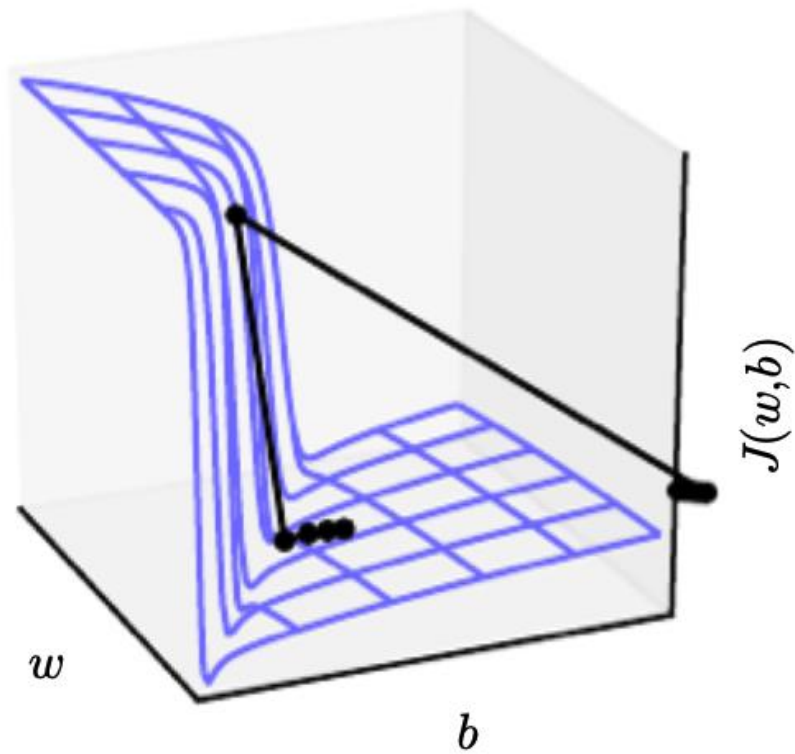
$$\begin{aligned} \frac{\partial L_T}{\partial h_t} &= \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_t} \\ &= \frac{\partial L_T}{\partial h_T} \prod_{k=t}^{T-1} \frac{\partial h_{k+1}}{\partial h_k} = \frac{\partial L_T}{\partial h_T} \prod_{k=t}^{T-1} D_{k+1} W_k^T \end{aligned}$$

Solving EVGP

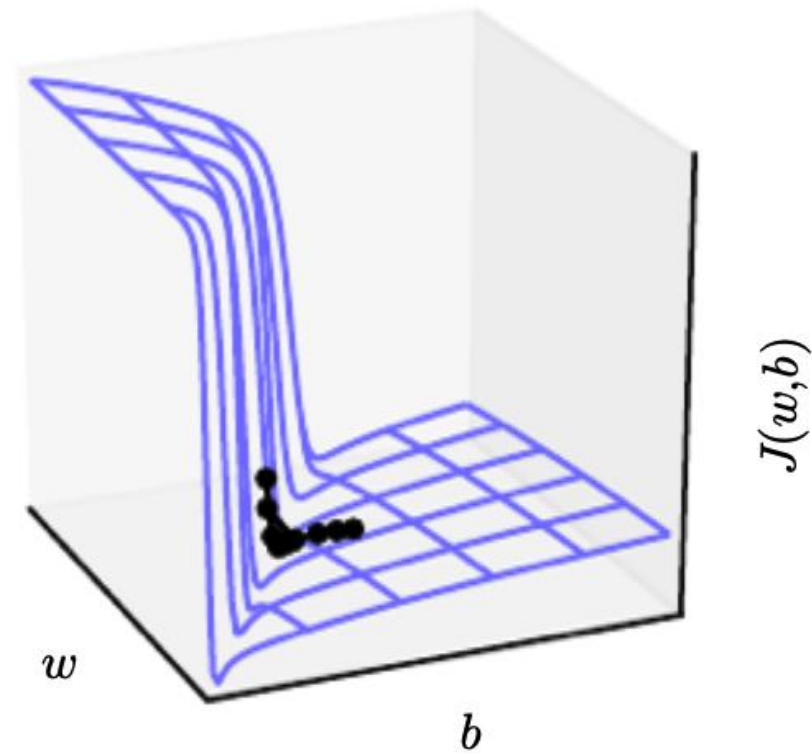


Gradient Clipping

Without clipping



With clipping



Gradient Clipping

re-scale the gradient just before the parameter update

$$\text{if } |g| > \alpha, \text{ then } g \leftarrow \frac{g\alpha}{|g|}$$

Stochastic Gradient Clipping

Denote by $\hat{g}_i$ the gradient of the minibatch i , then

$$g = \sum_i \hat{g}_i$$

What about the clipped gradient?

Neural Models for the Copy Task

